

Packaged solution projects

Some organizations acquire and adapt purchased *packaged solutions* (also called *commercial off-the-shelf*, or *COTS*, products) to meet their software needs, instead of building new systems from scratch. *Software as a service* (*SaaS*), or *cloud*, solutions are becoming increasingly available to meet software needs as well. Whether you're purchasing a package as part or all of the solution for a new project or implementing a solution in the cloud, you still need requirements. Requirements let you evaluate solution candidates so that you can select the most appropriate package, and then they let you adapt the package to meet your needs.

As Figure 22-1 shows, COTS packages typically need to be configured, integrated, and extended to work in the target environment. Some COTS products can be deployed out of the box with no additional work needed to make them usable. Most, though, require some customization. This could take the form of configuring the default product, creating integrations to other systems, and/or developing extensions to provide additional functionality that is not included in the COTS package. These activities all demand requirements.

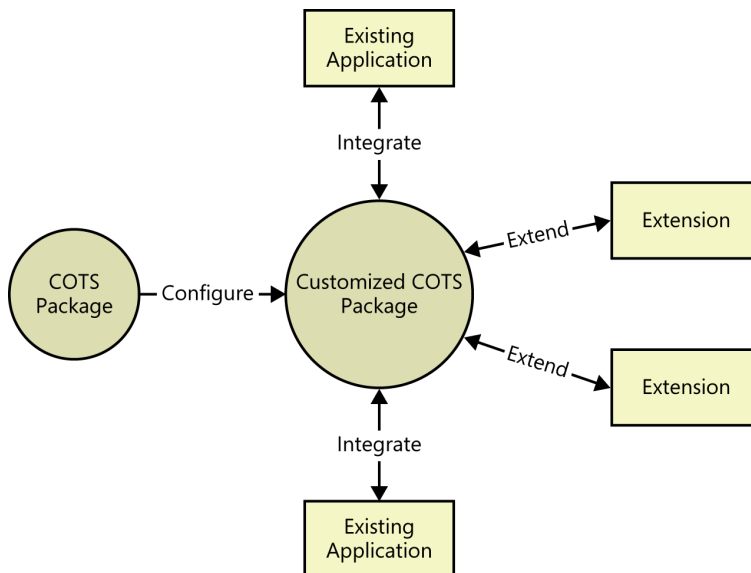


FIGURE 22-1 COTS packages can be configured, integrated into the existing application environment, and/or extended with new functionality.

This chapter discusses requirements for selecting and implementing packaged solutions. We do not distinguish between COTS and SaaS projects due to the similarity of the requirements activities involved. The decision to implement a packaged solution instead of custom developing a system is a matter of evaluating the cost-effectiveness of the two options and lies outside the scope of this book. If you're building a packaged solution to sell, the other chapters in the book are more relevant because those projects involve custom software development.

This chapter describes several ways to approach requirements definition when you plan to acquire a commercial package to meet your needs. We also provide suggestions for how to develop requirements to implement the packaged solution in your operating environment.

Requirements for selecting packaged solutions

COTS packages offer the acquiring organization less flexibility to meet requirements than custom (sometimes called *bespoke*) development does. You need to know which requested capabilities aren't negotiable and which you can adjust to fit within the package's constraints. The only way to choose the right packaged solution is to understand the business activities the package must let the users perform. Selecting packaged solutions entails identifying your requirements for the software, at least at a high level. The level of detail and effort you should put into specifying requirements for COTS selection depends on the expected package costs, the evaluation timeline, and the number of candidate solutions. Compare buying personal finance management software to buying a multimillion-dollar financial application for a 5,000-person company. You might only name the most important use cases in the first scenario, but write full use cases and develop data and quality requirements for a more extensive evaluation in the second.



One team needed to select packaged software to run a law office. They identified 20 tasks that users needed to perform using the software, which led to 10 features to be assessed while evaluating 4 candidate packages. The law partners knew they would have to create more detailed requirements to configure the software after they chose a package. However, a lightweight evaluation was appropriate for the package selection. In contrast, a team of 50 people worked together to develop detailed requirements for software to run a new semiconductor plant. There were only three candidate solutions to evaluate, but given the expected cost of the COTS software and its implementation, the company was willing to invest a lot in the selection process. They spent six months on the package selection alone.

Developing user requirements

Any package you choose must let users accomplish their task objectives, although different packaged solutions will do so in different ways. The majority of your requirements efforts for COTS acquisition should be focused at the user requirements level. Use cases and user stories work well for this purpose. Process models can also be used and might already exist in the organization. There's little point in specifying detailed functional requirements or designing a user interface, because the vendor (presumably) already did that.

It can also be helpful to list the features you need from the packaged solution. Identify the desired product features from an understanding of what users need to achieve with the solution and the business processes the package must enable. Suppose you have the following user story: “As a Research Manager, I need to review and approve new experiments before they are performed so that we don’t waste time and supplies on poorly designed experiments.” This user story helps identify the need for an approval workflow feature.

No packaged solution is likely to accommodate every use case you identify, so prioritize the user requirements or features. Trace them back to business requirements so you don’t waste time on unnecessary evaluation criteria. Distinguish capabilities that must be available on day one from those that can wait for future extensions and those that your users can live without, perhaps forever.

Considering business rules

Your requirements exploration should identify pertinent business rules to which the COTS product must conform. Can you configure the package to comply with your corporate policies, industry standards, and relevant regulations? How easily can you modify the configured package when these rules change? Focus on the most important business rules, because it can be time consuming to evaluate the implementation of all of the pertinent rules.

Some packages incorporate widely applicable business rules, such as income tax withholding computations or printed tax forms. Do you trust that these are implemented correctly? Will the package vendor provide you with timely software updates when those rules and computations change? Will they charge you for the updates? Will the vendor supply a list of the business rules the package implements? If the product implements any intrinsic business rules that don’t apply to you, can you disable, modify, or work around them? Does the vendor accept enhancement requests? If so, how are they prioritized?

Identifying data needs

You might need to define the data structures required to satisfy your user requirements and business rules, particularly if the new solution must be integrated into an ecosystem of existing applications. Look for major disconnects between your data model and the package vendor’s data model. Do not be distracted by data entities and attributes that are simply named differently in the COTS solution. Instead, recognize where entities or their attributes don’t exist in the packaged solution or have significantly different definitions from what you need, and then determine whether those entities can be handled in a different way for the solution to work.

Specify the reports that the COTS product must generate. Does it generate mandated reports in the correct formats? To what extent will the product let you customize its standard reports? Can you design new reports of your own to integrate with those that the vendor supplied?

Defining quality requirements

The quality attributes discussed in Chapter 14, “Beyond functionality,” are another vital aspect of user requirements that feeds into packaged solution selection. Explore at least the following attributes:

- **Performance** What maximum response times are acceptable for specific operations? Can the package handle the anticipated load of concurrent users and transaction throughput?
- **Usability** Does the package conform to any established user interface conventions? Is the interface similar to what the users experience in other applications already? How easily can your users learn to use the new package? Is training provided by the vendor included as part of the package’s cost?
- **Modifiability** How hard will it be for your developers to modify or extend the package to meet your specific needs? Does the package provide appropriate “hooks” (connection and extension points) and application programming interfaces for adding extensions? Will all those extensions stay in place when you install a new version of the package?
- **Interoperability** How easily can you integrate the package with your other enterprise applications? Does it use standard data interchange formats? Will it force you to upgrade any other third-party tools or infrastructure components because it doesn’t handle backward compatibility?
- **Integrity** Does the package safeguard data from loss, corruption, or unauthorized access?
- **Security** Does the package permit control over which users are allowed to access the system or use specific functions? Can you define the necessary user privilege levels? Particularly for SaaS solutions, evaluate the service level agreements very carefully against your requirements.

Evaluating solutions

Many commercial packages purport to provide canned solutions for some portion of your enterprise information-processing needs. Do some initial market research to determine which packages are viable candidates deserving further consideration. Then you can use the requirements you identified as evaluation criteria in an informed COTS software selection process.

One evaluation approach includes the following sequence of activities (Lawlis et al. 2001):

1. Weight your requirements on a scale of 1 to 10 to distinguish their importance.
2. Rate each candidate package as to how well it satisfies each requirement. Use a rating of 1 for full satisfaction, 0.5 for partial satisfaction, and 0 for no coverage. You can find the information to make this assessment from product literature, a vendor’s response to a request for proposal (RFP), or direct examination of the product. Keep in mind that an RFP is an invitation to bid on a project and might not provide information that reflects how you intend to use the product. Direct examination is necessary for high-priority requirements.
3. Calculate the score for each candidate based on the weight you gave each factor, to see which products appear to best fit your needs.

4. Evaluate product cost, vendor experience and viability, vendor support for the product, external interfaces that will enable extension and integration, and compliance with any technology requirements or constraints for your environment. Cost will be a selection factor, but evaluate the candidates initially without considering their cost.

You might consider which requirements are *not* met by any of the candidate packages and will require you to develop extensions. These can add significant costs to the COTS implementation and should be considered in the evaluation process.



Recently, my organization wanted to select a requirements management tool that—among other capabilities—allowed users to work offline and synchronize to the master version of the requirements when the users went back online (Beatty and Ferrari 2011). We suspected that no tools on the market would offer a good solution for this. We included this capability in our evaluation to ensure that we uncovered any solutions that did offer it. If we didn't find one, we would know that it was a capability we'd have to implement as an extension to the selected package. Alternatively, we'd need to change our process for editing requirements.

Another evaluation approach is to determine whether—and how well—the package will let the users perform their tasks by deriving tests from the high-priority use cases. Include tests that explore how the system handles significant exception conditions that might arise. Walk through those tests to see how the candidate packages handle them. A similar approach is to run the COTS product through a suite of scenarios that represent the expected usage patterns, which is called an *operational profile* (Musa 1999).

Trap If you don't have at least one person whose involvement spans all of the evaluations, there is no assurance that comparable interpretations of the features and scores were used.

The output of the evaluation process is typically an evaluation matrix with the selection requirements in the rows and various solutions' scores for each of those requirements in the columns. Figure 22-2 shows part of a sample evaluation matrix for a requirements management tool.

ID	Use Case	Feature	Priority	Tool 1	Tool 1 Comments
1	BA adds new requirements individually	Add new requirement	10	1	
2	BA adds new requirements individually	Automatically create unique ID for each requirement	10	1	
3	BA adds new requirements individually	Document a requirement with rich text formatting	3	0	Only supports text with no formatting.
5	BA models requirements	Describe a requirement with an image directly in the database	6	0.5	Need to do a workaround for this where you link to the resource outside the tool but store the link in the database.
6	BA models requirements	Describe a requirement with an embedded document in the database	8	1	
7	BA links existing documentation to requirements	Link requirements to actual documents in a SharePoint location	4	1	
9	BA adds a bulk of new requirements at once	Batch import structured data as new requirements from Excel	5	1	Batch import is supported and provides support for importing customized Excel files.

FIGURE 22-2 A sample of a packaged solution evaluation matrix for a requirements management tool.



Multi-stage evaluation

When I wrote the requirements for selecting a requirements management tool for our own consulting teams to use, I worked with the teams to identify the user classes and use cases for the tool. Although the primary users were business analysts, there were also a few use cases for managers, developers, and customers. I defined use cases by name and used my familiarity with the use cases to identify desired features. I created a traceability matrix to minimize the likelihood that any use cases or features would be missed.

We started with 200 features and 60 vendor choices, which were far too many for our evaluation timeline. We did a first-pass evaluation to eliminate most of the candidate tools. Our first pass considered only 30 features that we deemed the most important or most likely to distinguish tools from one another. This initial evaluation narrowed our search to 16 tool choices. Then we evaluated those 16 against the full set of 200 features. This detailed second-level evaluation resulted in a list of five closely ranked tools, all of which would clearly meet our needs.

In addition to an objective analysis, it's a good idea to evaluate candidate packages by using a real project, not just the tutorial project that comes with the product. We ended up adding a third level of evaluation to actually try each of those five tools on real projects so we could see which one most closely reflected the evaluation scores in practice. The third phase of the evaluation allowed us to select our favorite tool from the high-scoring ones.

Requirements for implementing packaged solutions

After you decide to implement a selected packaged solution, there is still more requirements work to do. Figure 22-3 shows that the spectrum of effort required to make a packaged solution useful ranges from using the package as is, right out of the box, to performing considerable requirements specification and software development for extensions. Table 22-1 describes these four types of COTS package implementations, which are not mutually exclusive. Any of these implementations might also require making infrastructure changes in the operating environment, such as upgrading operating systems or other software components that interact with the package.

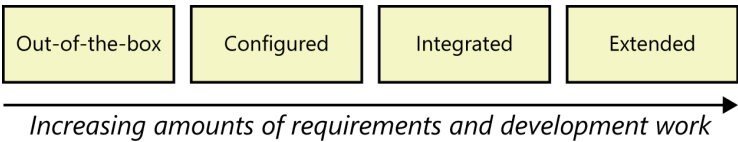


FIGURE 22-3 A spectrum of implementation effort for packaged solutions.

TABLE 22-1 COTS package implementation approaches

Type of COTS implementation	Description
Out-of-the-box	Install the software and use it as is.
Configured	Adjust settings in the software to suit your needs without writing new code.
Integrated	Connect the package to existing systems in your application ecosystem; usually requires some custom code.
Extended	Develop additional functionality with custom code to enhance the package's capabilities to close needs gaps.

One advantage of purchasing a COTS solution is that it might provide useful capabilities that you hadn't originally sought. You typically select the package based on what you know you need. However, during implementation, you might discover valuable features that you hadn't even thought of. This can change the amount of work needed to install the package to exploit the additional features.

Configuration requirements

Sometimes you can use a package just as it comes from the vendor. More often, you'll need to adjust various configuration parameters in the package to better meet your needs. Configuration requirements are essential to most successful COTS implementations. One approach is to define configuration requirements for one process flow, use case, or user story at a time. Walk through user manuals for the purchased system to learn how to execute a specific task, looking for settings that need to be configured to suit your environment. Consider the full set of business rules when you are configuring the system, not just those you examined during the selection process. It might be helpful to create decision tables and decision trees to model these requirements. Many COTS solutions come with predefined mechanisms to specify roles and permissions. Use a roles and permissions matrix, such as the one shown in Figure 9-2 in Chapter 9, "Playing by the rules," to define which roles to create and what permissions those roles should have.

Integration requirements

Unless the packaged solution is used in a standalone mode, you'll need to integrate it into your application environment. This integration involves understanding the external interfaces the package will present to each of the other applications with which it must interact. Precisely specify the requirements for interchanging data and services between the package and other components in your environment. You will likely have to create some custom code to make all the parts fit together. This code could take the form of:

- Adapters that modify interfaces or add missing functionality.
- Firewalls that isolate the COTS software from other parts of the enterprise.
- Wrappers that intercept inputs to and outputs from the package and modify the data as necessary to be used on the other side of the interface (NASA 2009).

Extension requirements

One common goal of COTS implementations is to minimize customizations to the solution. Otherwise, you should just custom build the application yourself. In most COTS projects, though, there will be gaps between what the organization needs and what the package delivers. For each such gap, decide whether to ignore it (remove the requirement and just live with the tool); change how you do something outside the solution (modify the business process); or build something to bridge the gap (extend the solution). If you are extending the COTS solution, you'll need to fully specify the requirements for those new capabilities just as you would for any new product development. If you are implementing a COTS solution to replace an older system, look at the practices related to replacing a system that were discussed in Chapter 21, "Enhancement and replacement projects." While analyzing the requirements for any components to be added, assess whether they could negatively affect any existing elements or workflows in the package.

Data requirements

Begin with the data requirements used in the selection process. Map data entities and attributes from your existing data dictionary to the COTS entities and attributes. There will likely be areas where the solution doesn't handle some of your existing data entities or attributes. As with functional gaps, you'll need to decide how to handle data gaps, typically by adding attributes or repurposing an existing data structure in the COTS solution. Otherwise, when you convert data from any existing systems into the COTS solution, you will likely lose any data that was not properly mapped. Use report tables to specify requirements for deploying existing or new reports, as described in Chapter 13, "Specifying data requirements." Many COTS packages will provide some standard report templates to start with.

Business process changes

COTS packages are usually selected because implementing and maintaining them is expected to be less expensive than building custom software. Organizations need to be prepared to adapt their business processes to the package's workflow capabilities and limitations. This is different from most development projects where the software is designed specifically to accommodate existing or planned processes. In fact, a COTS solution that can be fully configured to meet your existing processes is likely to be expensive and complex. The more buttons and knobs you can adjust, the harder it is to configure. You need to strike a balance between implementing all of the desired user functionality and only what the COTS product offers out of the box (Chung, Hooper, and Huynh 2001).

Start with the user requirements identified during the selection process. Develop use cases or swimlane diagrams to understand how the tasks will change when users execute their tasks in the COTS solution. Users might resist the new packaged solution because it looks or behaves differently than their existing systems, so involve them early in this process. Users are more willing to accept the new solution if they contributed to shaping the necessary changes in their business processes.



My team implemented a packaged solution for an insurance company to let them meet new compliance requirements. We started by modeling the as-is business processes. Then we studied the package's manuals to learn basic information about how to use the product. Based on the as-is models, we created to-be business processes to reflect how the users would complete their tasks using the COTS solution. We also created a data dictionary for their existing system and added a column to reflect the mapped field in the COTS solution. The users helped develop all of these work products, so they weren't surprised by the new system when it was deployed.

Common challenges with packaged solutions

The following are common challenges that you might encounter when selecting or implementing a packaged solution:

- **Too many candidates** There might be many solutions on the market that meet your needs at first glance. Select a short list of criteria to narrow the candidate list to a few top choices for a more refined evaluation.
- **Too many evaluation criteria** It might be hard to focus the evaluation criteria to only the most important ones without doing in-depth requirements specification. Use business objectives to help select the most important requirements as criteria. If you narrow the candidate package choices down to only a few, you can evaluate them against a long list of criteria.
- **Vendor misrepresents package capabilities** In the typical packaged software purchasing process, the vendor sales staff sells their solution to the customer organization's decision makers, and then engages a technical implementation team to provide in-depth knowledge about the product. That in-depth knowledge might prove to conflict somewhat with the customer's understanding of the product's capabilities based on the sales pitch. It's a good

idea to ask to have a vendor technical specialist participate during the sales cycle. Determine whether you can have a healthy relationship with the vendor that enables both parties to be successful. The vendor is your business partner, so make sure they can play that role constructively.

- **Incorrect solution expectations** Sometimes a solution looks great during vendor demos, but it doesn't work like you expect after installation. To avoid this, during the selection process, have the vendor walk through your actual use cases so you can see how well the solution matches your expectations.
- **Users reject the solution** Just because an organization bought the software, there is no guarantee that the users will be receptive to it. As with all software development projects, engage users in the selection process or early in the implementation to make sure their needs are clearly understood and addressed to the extent possible. Expectation management is an important part of successful packaged solution implementation.

Buying, configuring, and extending a commercial software package often is a sensible business alternative to building a custom solution. Packages can provide a lot of flexibility, but at the same time they come with built-in limitations and constraints. You don't want to have to pay for a lot of features that your organization doesn't need. Nor do you want to build a fragile structure of extensions and integrations that might break with the next release of the package from the vendor. A careful package selection and implementation process will help you find the optimum balance of capability, usability, extensibility, and maintainability in a commercial packaged software solution.